

UDP Chat Application

Student Name: Kushalkumar Thakar

Student Id: 5155617

Algoma University

Department of Computer Science and Mathematics

Final Report for COSC 5906

Brampton, Ontario, Canada

Date: 8 Aug, 2026

I. INTRODUCTION

This project implements a peer-peer chat application for iOS using swift and swiftUI, Two running clients exchange text messages as user Datagram Protocol Packet. Apple's Network framework performance the low-level networking, Bonjour discovers nearby peers automatically, and Wireshark captures the resulting traffic for the protocol analysis.

Most production chat system use TCP, WebSockets, or another reliable transport, This project intentionally uses UDP so that connectionless communication, low protocol overhead, independent datagrams, and the absence of the built-in reliable can be observed directly.

Understand UDP socket programming using NWListener and NWConnection. Send and receive complete UDP datagrams between two clients. Study IP addressing, source ports, destination port, packet length, and payload data. Bonjour to automatically discover the addresses and ports. Capture packets with Wireshark and compare UDP behavior with TCP.

II. LITERATURE REVIEW

Apple Inc. "Network Framework Documentation": This documentation outlines the modern swift-native approach to managing network connections. It details how NWConnection and NWListener replace legacy socket APIs, providing a robust interface for both TCP and UDP protocols.

Cheshire S., & Ktochmal M "Zero Configuration Networking": This provides a comprehensive overview of Bonjour and mDNS. It explains how devices advertise service over local subnet, which informs our decision to use the udp service type for peer discovery.

Hoffman L. "Mastering SwiftUI": This work discusses declarative user interface design in iOS. The concepts covered are essential for understanding how to bind the asynchronous network discovery state to the reactive SwiftUI views.

III. MAIN GOAL

The main goal is to design, implement and empirically verify an iOS application in which two or more instances on the same local link discover one another through Bonjour service discovery and exchange chat messages as UDP datagram, and in which the resulting traffic remains directly observable in a packet analyzer.

Four objective follows:

1. Each instance advertises itself as an instance of `_udpchat._udp` on a port chosen by the operating system rather than fixed in code, while simultaneously browsing for others and maintaining a live peer list.
2. A message is encoded as a self-describing JSON packet and sent as a single datagram addresses to a discovered service endpoint rather than to a user supplied address.
3. The receiver reconstructs exactly one message from exactly one datagram, rejects foreign or unsupported packets, and suppresses duplicates by identifier.
4. Discovery and message traffic are captured and identified in Wireshark using filters that do not depends on a fixed port.

This project succeeds if all four are met on at least two independent pairings of the devices, including one involving physical hardware.

IV. RATIONALE

UDP chat application makes a normally invisible layer of networking directly observable. Most mobile development happens well above the wire: a request goes to a high-level library, a response

arrives, the addressing and framing between are never seen. A chat application built on raw UDP and mDNS inverts that every design decision has a visible consequence in a packet capture, and every packet trace back to a specific line of code. The topic also matters beyond its instructional value, since communication of infrastructure is significant wherever infrastructure is absent, saturated or not trusted. This project yields a documented reference implementation of Bonjour discovery combined with dynamic port UDP messaging on iOS.

V. SCOPE

The project covers the design, implementation, and testing of a single-view iOS application of Swift files and it's property list configuration, together with the functional testing across simulator and device pairing and packet level verification in Wireshark.

Discovery is limited to Bonjour over multicast DNS on a single local link using the type `_udpchat._udp`, and transport to UDP datagram each carrying one JSON encoded packet, with one selected peer at a time.

Several limitation follows. The most significant is the absence of the confidentiality and authentication. Datagram are plain JSON, and any device on the link can read them, forge them or advertise a competing instance of service type. Secondly, the environment is not fully controllable multi cast is filtered on many managed network, client isolation prevents peer-to-peer traffic on guest wifi, and VPN software commonly surpasses local-link traffic. Third, evaluation is functional rather than statical, and a declined local network permission cause discovery to fail silently from the application's perspective.

VI. TIMETABLE

Days	Goal	Task description
1 - 2	Literature review	Review literature and draft requirements
3 - 4	SwiftUI View	Build view in SwiftUI
5 - 6	Network Setup	Configure NWListener to open an assigned UDP port
7 - 8	Service Discovery	Advertise as <code>__udpchar._udp</code> and discover instances.
8 - 9	Message Handling	Implement JSON encoding/decoding and NWConnection
10 - 11	Testing	Test using Simulator and Wireshark

TABLE I. Project Timetable

VII. METHODS

A. Materials

This project is a pure software artifact. It uses an Apple silicon Mac with Xcode 15 or later, the target is two iPhone simulator or iPhone devices with iOS 17 or later. The application is written in Swift with SwiftUI, and uses the Apple Network Framework. Wireshark is used to capture all data transfers between connected devices.

B. Technologies Used

Technology	Role in the Project
Swift	Message Model, JSON encoding, networking logic, and error handling
SwiftUI	User input, peer selection, and message display
Network framework	UDP listener, browser, service endpoint, send and receive operations
Bonjour	Automatic local-network peer and port discovery
Xcode	Development and two-client testing
Wireshark	Packet capture and examination of protocol fields

TABLE II. Technologies

C. UDP Fundamentals

UDP is commonly used for DNS, real-time voice and video, online games, telemetry, and other applications where delay may matter more than perfect delivery. In this chat project, one encoded chat message is placed in one UDP datagram.

Advantages	Limitations
No connection handshake	No delivery guarantee
Small 8-byte transport header	No ordering guarantee
Low latency and protocol overhead	No automatic retransmission
Preserves message boundaries	No flow or congestion control

TABLE III. UDP Fundamentals

D. System Architecture and Data Model

The application has five layers: A SwiftUI view hierarchy, A single observable service object owning all networking state, A serialization layer converting between the packet structure and JSON, The Network framework providing listener, browser, and connections, The operating system's mDNS responder and UDP/IP stack. The instance communicates only through the shared local link, no server, no coordinator, no shared state beyond what is advertised over multicast DNS. The observable object patterns enforce the separation, The service exposes only start, stop and send.

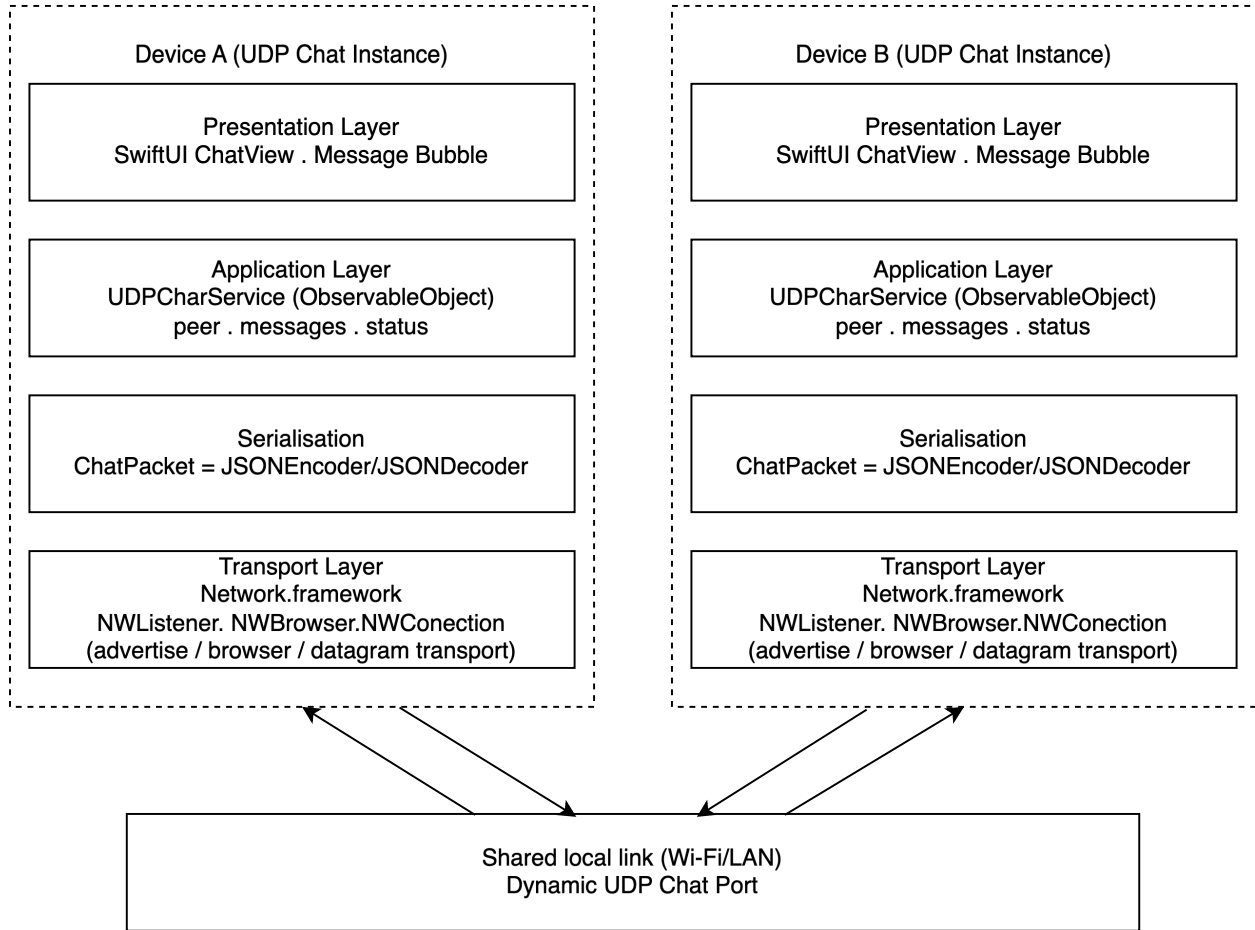


Fig. 1. Five Layers

In Fig. 1 Layered architecture of two communicating instance. No component is shared between the devices. All coordination takes place through multicast DNS on the local link.

E. Swift Networking Implementation

It follow MVVM(Model - View - ViewModel) architecture.

View: Display nearby peers and chat messages, and collects the user's text input.

Application state: Track the selected peer, networking status, and message list.

UDPChatService: Creates the listener and browser, sends and receives Data, decode packets, and publishes results.

Message model: Stores the message ID, sender, text, timestamp, and direction.

Peer endpoint: Represents the Bonjour services used by NWConnection to resolve the remote address and port.

E.1 Automatic Listener and Bonjour Advertisement

The final implementation does not ask the user for a local port. NWListener is created without a specific port, so the opening system selects an available UDP port. The listener then advertise the service type `_udpchat._udp` through Bonjour.

```

let listener = try NWListener(using: .udp)
listener.newConnectionHandler = { connection in
    connection.start(queue: self.queue)
    self.receiveNextMessage(on: connection)
}

listener.start(queue: self.queue)

```

Fig. 2. Listener

E.2 Peer Discovery

NWBrowser searches for the same Bonjour service type. Each result contains an `NWEndpoint.service` value. Passing that endpoint to `NWConnection` lets the Network framework resolve the peer's IP address, dynamic UDP port, and interface automatically.

```

let descriptor = NWBrowser.Descriptor.bonjour(type: "_udpchat._udp",
                                              domain: nil)
let browser = NWBrowser(for: descriptor, using: .udp)
let connection = NWConnection(to: peer.endpoint, using: .udp)

```

Fig. 3. Peer Discovery

E.3 Sending and Receiving Datagrams

```

let messageData = text.data(using: .utf8)
connection.send(
    content: messageData,
    completion: .contentProcessed { error in
        if let error {
            print("Send error: \(error)")
        }
    }
)

```

Fig. 4. Send Data

```

connection.receiveMessage { data, _, _, error in
    if let data, let text = String(data: data, encoding: .utf8) {
        DispatchQueue.main.async {
            messages.append(text)
        }
    }
}

```

Fig. 5. Receive Data

isCompleter: true identified the content as a complete UDP message. `receiveMessage` returns one datagram at a time. Network callback run asynchronously, so updates to `@Published` properties are dispatched to the main thread before SwiftUI is refreshed.

E.4 Application Packet Format

The app uses JSON so that the payload can be decoded by the receiver and inspected easily in Wireshark. A packet contains an application identifier, protocol version, UUID sender, text and timestamp.

F. Message Flow and Network Addressing

F.1 End-to-End Message Flow

1. The user select a discovered peer and enters a message.
2. SwiftUI passes the message to `UDPChatService`.
3. The service creates a `ChatPacket` and `JSONEncoder` converts it into `Data`.
4. `NWConnection` resolves the Bonjour endpoint and sends one UDP datagram.
5. The receiving `NWListener` accepts the incoming UDP flow.
6. `ReceiveMessage` returns the datagram and `JSONDecoder` reconstructs the packet.
7. The received message is appended on the main thread and displayed in SwiftUI.

F.2 Sequence Diagram

In Fig. 6 Steps 1-5 are the mDNS exchange. Step 6 is the chat datagram, addressed to the discovered service endpoint rather than an IP address. Dashed arrows indicate passive observation.

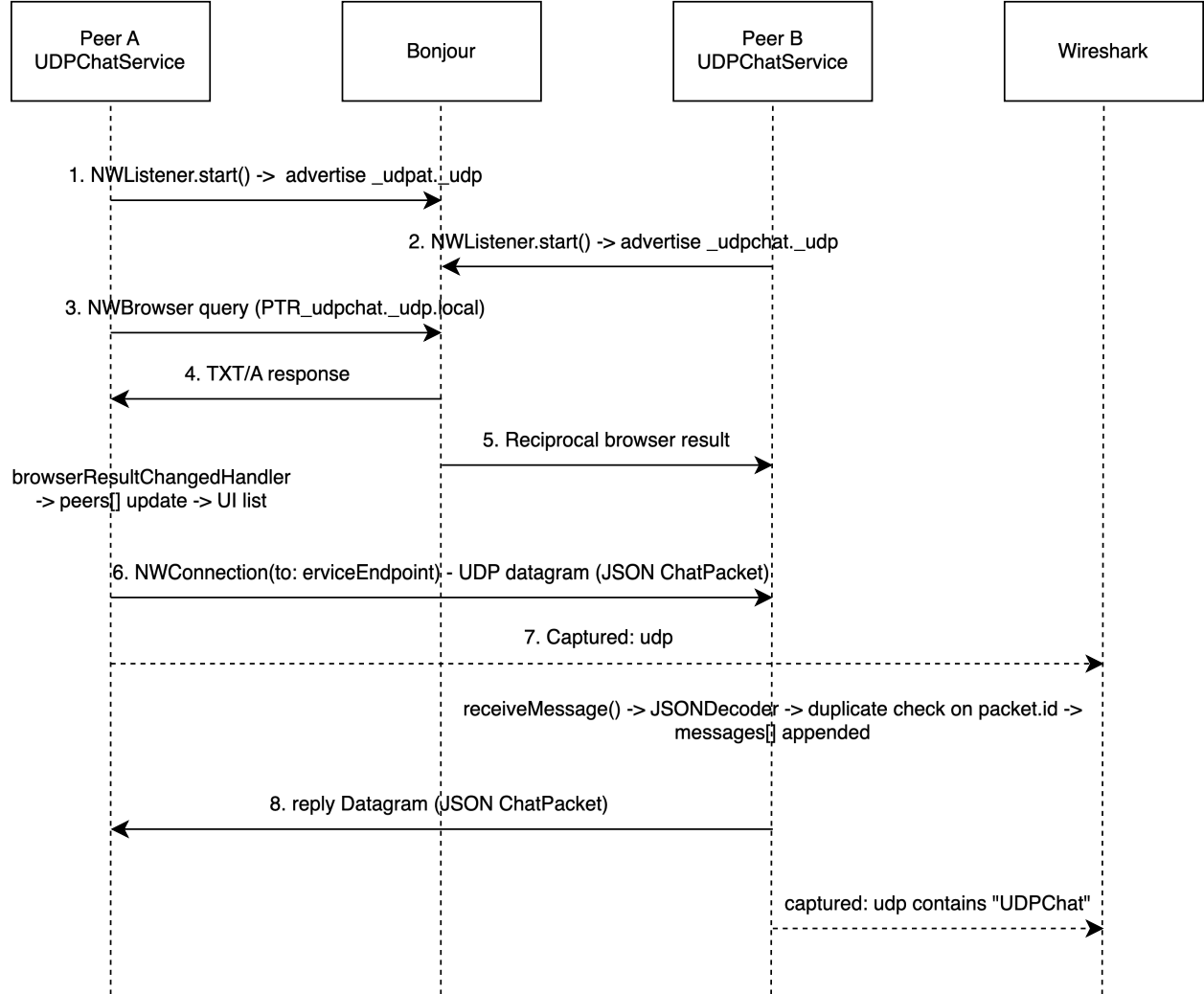


Fig. 6. Discovery And Message Sequence

Client A	Enter text	SwiftUI
SwiftUI	Call send(text:to:)	UDPChatService
UDPChatService	JSON Data in UDP datagram	Network
Network	Connectionless packet delivery	Peer listener
Peer listener	receiveMessage + decode	Peer service
Peer service	Publish received message	Client B UI

TABLE IV. Sequence

E.3 IP Addresses and Ports

An IP address identifies a device or network interface, while a port number identifies a particular application endpoint on that device. Using Bonjour, these are discovered and managed automatically.

Endpoint	Manual Example	Final implementation
Client A	172.28.1.23	Bonjour service + Dynamic UDP port
Client B	172.28.1.23	Bonjour service + Dynamic UDP port
Packet direction	23	Resolved service endpoint -> listener port

TABLE V. IP Address And Ports

G. Wireshark Packet Capture and Analysis

The application was tested with two iOS simulator clients. Wireshark captured the traffic from the Mac loopback interface(lo0). Because the Bonjour version uses dynamic ports, the application identifier in the JSON payload was used as the display filter. filter: udp contains "UDPChat"

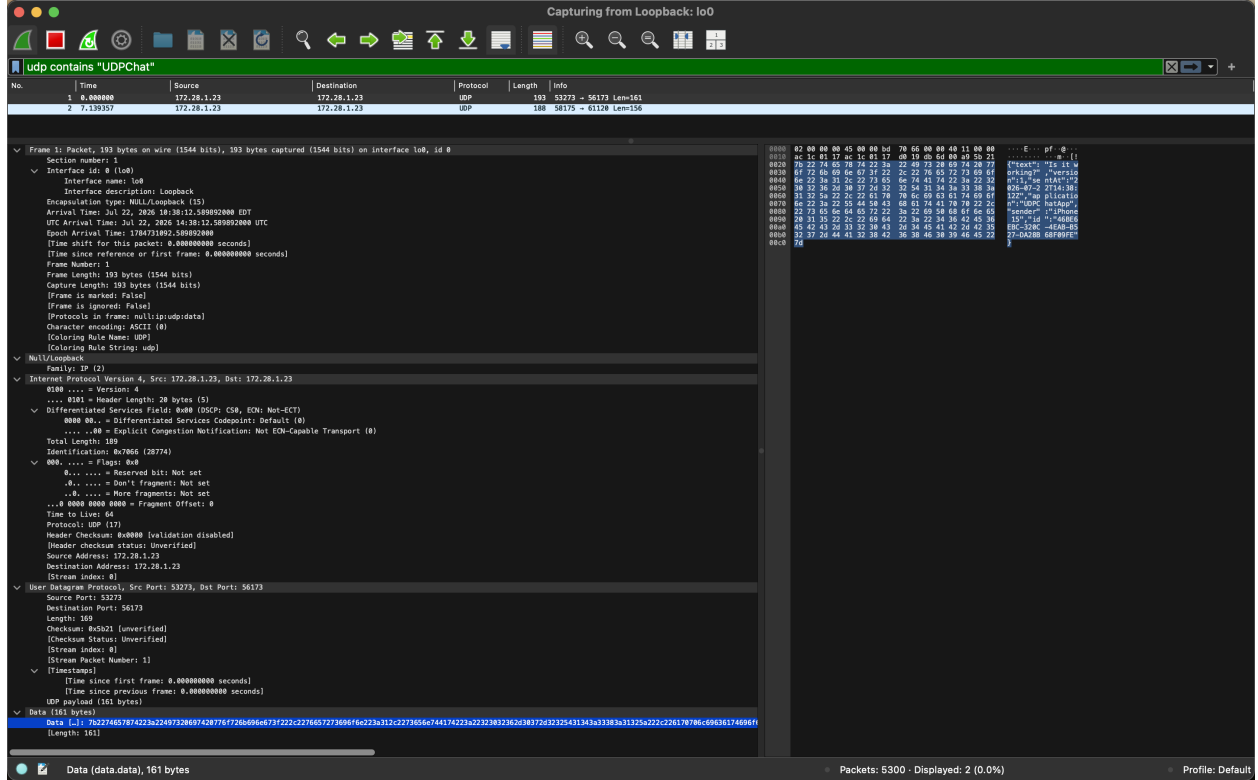


Fig. 7. Simulator A Output

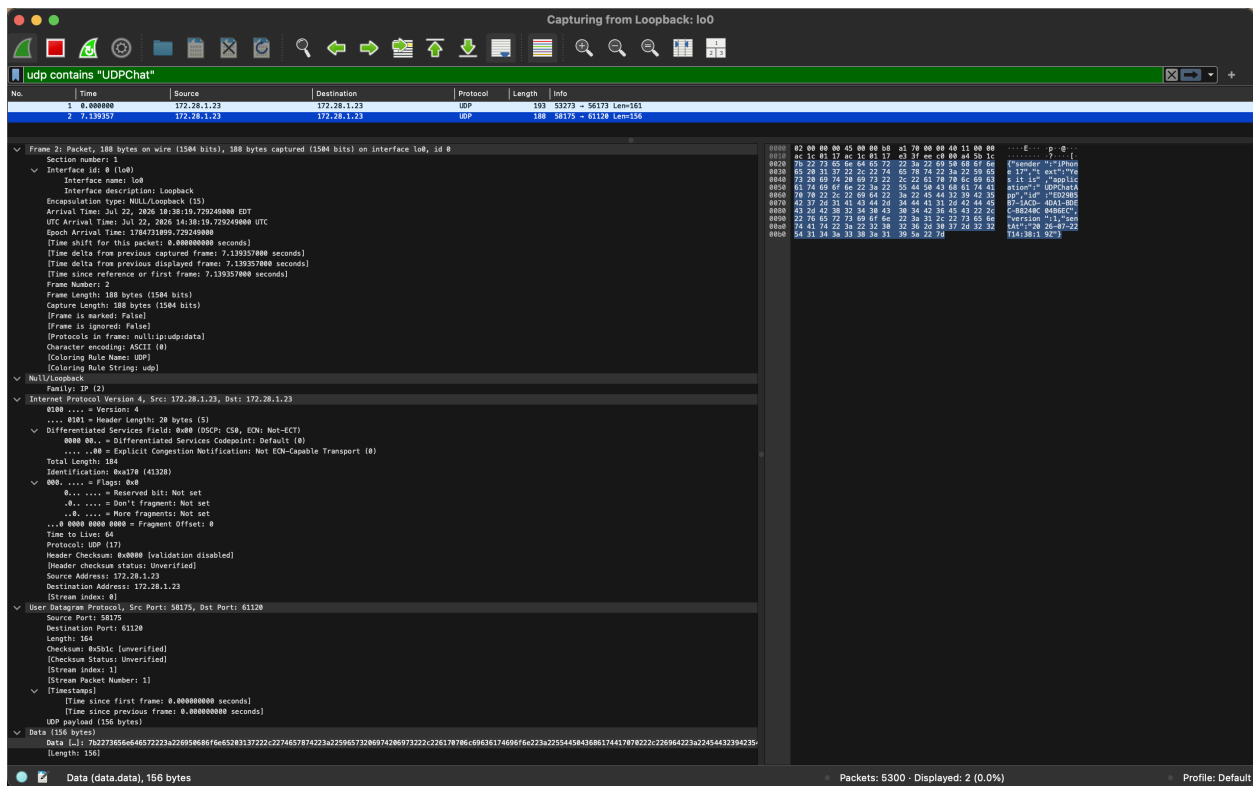


Fig. 8. Simulator B Output

VIII. RESULTS AND CONCLUSION

A. SwiftUI Output

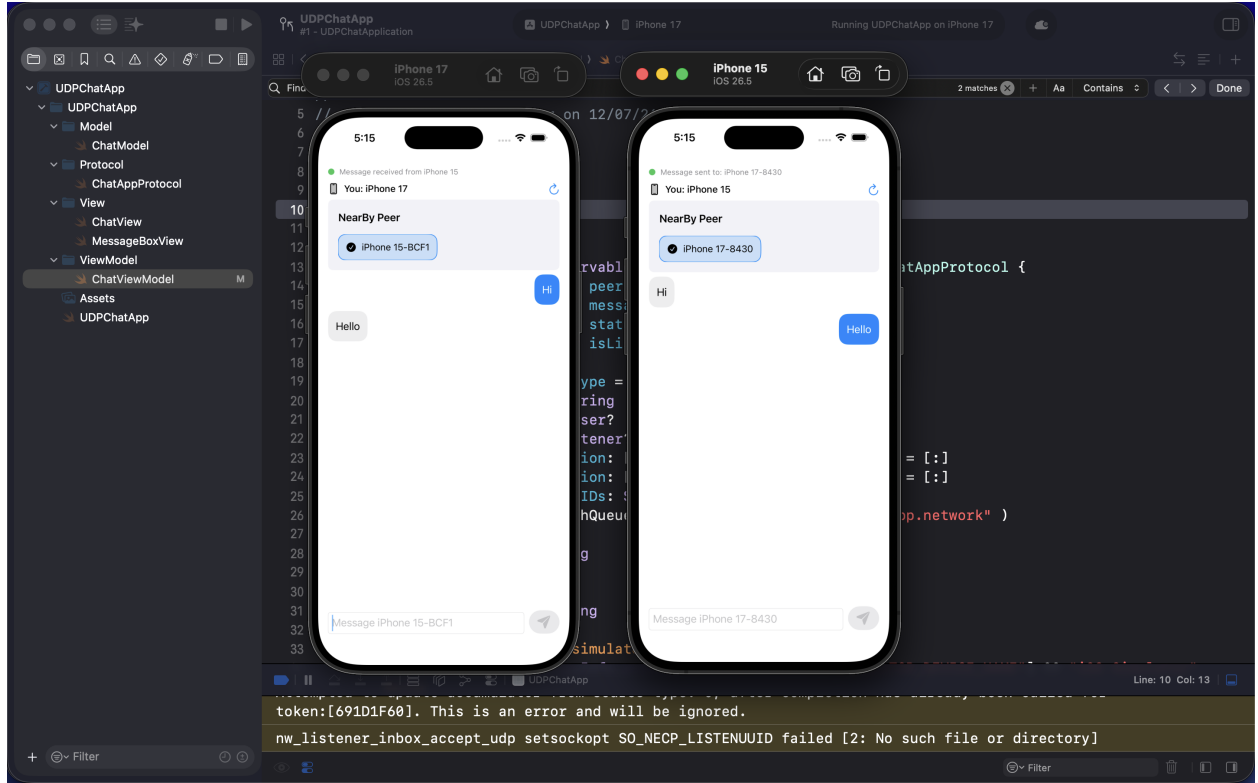


Fig. 9. SwiftUI Output

B. Observed packet Results

Test	Expected Behaviour	Outcome
Launche both instance	Each lists the other, neither list itself	Pass
Send a message	Sent bubble locally, received bubble on the peer	Pass
Terminate one instance	Withdrawn peer disappers, selection falls back	Pass
Refuse permission	No peers found, status report none found	Pass
Foreign datagram	Valid JSON with a different tag rejected	Pass
Duplicate identifier	Repeat of a seen identifier discarded silently	Pass
Packet capture	Traffic located by filter, Payload legible	Pass
VPN active on device	Discovery fails, enviromental limitation	Fail(external)

TABLE VI. Functional Test

Field	Packet 1	Packet 2
Interface	Loopback lo0	Loopback lo0
Source -> destination IP	172.28.1.23 -> 172.28.1.23	172.28.1.23 -> 172.28.1.23
Source -> destination port	53273 -> 56173	58175 -> 61120
UDP Length	169 bytes	164 bytes
Paylaod	161-byte JSON data	156-byte JSON data

TABLE VII. Result

C. Limitation and Future Improvements

No guaranteed delivery, acknowledgment, retransmission, encryption, or authentication.
No sequence numbers, duplicate protection, persistent history, or reliable NAT traversal.
Possible improvement include acknowledgment, retry timers, sequence numbers, duplicate detection, encryption, group chat, file transfer, and a TCP version for comparison.

D. Conclusion

The project demonstrates connectionless peer-to-peer communication using Swift and Apple's Network framework. Bonjour improves usability by discovering peers and resolving dynamic port automatically. Wireshark verifies the IP and UDP fields and shows that each chat message is carried as a separate readable datagram. The experiment highlight UDP's speed and low overhead while also showing why reliable production chat systems generally require additional application-level mechanism or TCP.

IX. REFERENCE

<https://developer.apple.com/documentation/network>
<https://developer.apple.com/documentation/swiftui>

X. APPENDICES

Source Code: Click to access Github link of my UDP Chat Application

Note: To keep everything on Github I have also uploaded my PPT and this PDF in the above Github link.